AI Engineering and Claude Architect · Part IV, Java

AI in Java, Lesson 5

A RAG Service With pgvector

Search filtered by department after retrieval. It looked fine in tests. In production, with a small team's narrow filter, the top 50 results held nothing they could see, so the assistant said "I don't know" about their own documents.

Module	AI08 of 13
Lesson	5 of 8
Prerequisites	AI04 L3–L4, AI05 L4, L7, AI08 L1
Exam relevance	Out of scope. Context assembly supports TS 5.1
Reading time	~30 minutes

Contents

1. A RAG Service With pgvector

Lesson 5 of 8 · AI in Java

A RAG Service With pgvector

Key idea

The RAG service's job is mostly decisions that aren't about the model: check permissions first, fail closed, filter inside the search, fuse two rankings, and skip the model when there's no evidence. All of these can be tested with no key and no database. **Search quality can't be, so that remains for a real evaluation on real data.**

> *Before: AI05 L7's design, AI04 L4's pgvector schema. Teaches: that query path as a Spring service, behind ports, with tests. Used by: AI08 L8.*

The hook: decisions you can test

Real output from `RagServiceTest`, with a scripted model and an in-memory search:

```
asha asks about quarter close      ANSWER       sources=[fin-1] modelCalls=1
ravi asks about quarter close      NOT_FOUND    sources=[]      modelCalls=1
directory can't answer             UNAVAILABLE  sources=[]      modelCalls=0
```

Line	Why it matters
Asha	Has the group, so she gets the answer
Ravi	Same question, no access: `modelCalls` stayed at 1, so **the model was never called** for Ravi
Directory down	**Fails closed**, with no search and no model call

What's real here, and what isn't

Real: the service's logic, and these tests. Not run: `PgVectorSearch`, because no Postgres is available here. The "meaning" search in the test is word overlap with a synonym table, so nothing here measures retrieval quality. The SQL follows AI04 L4's schema and pgvector's docs, and it needs testing on your version.

Intuition: a records office

A clerk first checks your badge, and if the badge system is down, the office closes. Only then does anyone search, and only through files you're cleared for. If nothing's found, you're told so at once, with no need to wake the expert upstairs.

Mechanism: the query path

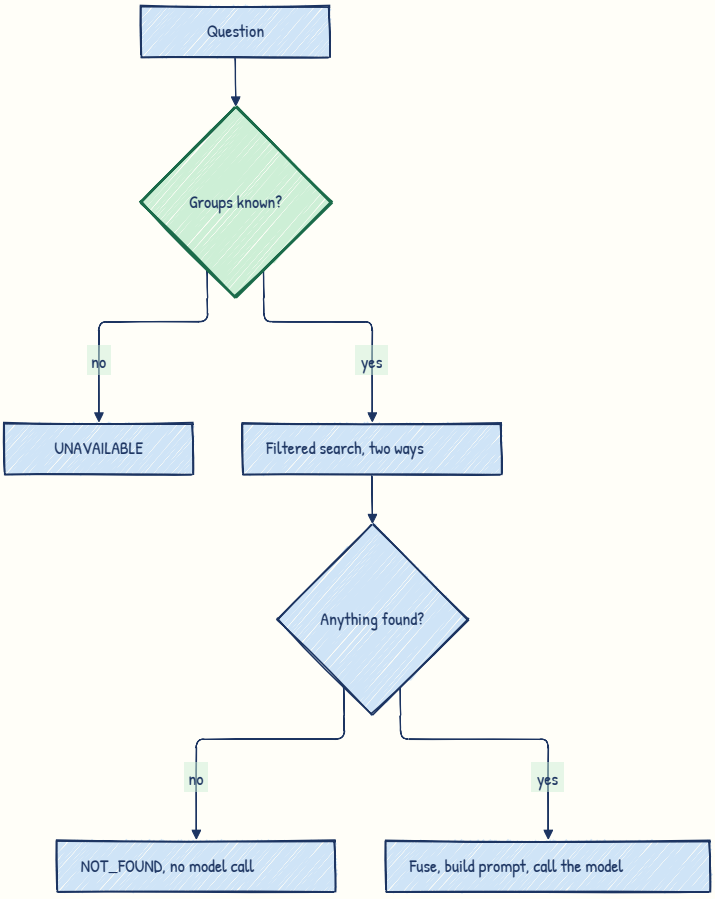


Figure 1. The query path as built. Every early exit avoids a model call, and permissions sit in front of everything.

From `RagService.java`:

```
Set<String> groups;
try {
    groups = directory.groupsOf(employee);
} catch (RuntimeException unavailable) {
```

The ports

```
public interface ChunkSearch {
    List<Hit> byMeaning(String query, Set<String> groups, int limit);

    List<Hit> byKeywords(String query, Set<String> groups, int limit);
}
```

Groups are a required argument, so a caller can't search without them. That's the design doing the work.

Filtering inside the query

Here's `PgVectorSearch.java`, compiled and not run. The filter is in the `WHERE` clause, so the database returns the best **permitted** rows, rather than the best rows, filtered afterwards:

```
SELECT id, source, published, body FROM chunks
WHERE allowed_groups && ?::text[]
ORDER BY embedding <=> ?::vector
LIMIT ?""", (row, n) -> chunk(row), array(groups), vector, limit));
```

Concern	Detail	From
Filter with an approximate index	Can return too few rows. Use iterative scans	AI04 L3
Keyword search	Postgres full text, here with `websearch_to_tsquery`	AI04 L4
Array building	My `array()` helper joins strings, which breaks if a group name has a comma. Use `Connection.createArrayOf` in real code	My own review

Fusing two rankings

Reciprocal Rank Fusion, from AI04 L3. Only ranks matter, so there's no score scaling:

```
scores.merge(hit.chunk().id(), 1.0 / (RRF_K + hit.rank()), Double::sum);
```

A real service adds reranking here. It's not included, since AI04 couldn't confirm a Java API shape for one, and I didn't add one.

Assembling the prompt

Documents first, the question last, with sources and dates (AI05 L4). Real output:

```
<documents>
<document index="1"><source>refunds.md</source><published>2026-02-01</published>
<document_content>A refund takes 5 to 10 days.</document_content></document>
</documents>
Question: How do I get my money back on a refund?
```

Line breaks are slightly adjusted. The system prompt tells it to answer only from these documents, say so if they're silent, and report disagreements with sources and dates.

This service doesn't use the Citations API yet, because the port takes plain text. `CitedAnswers.java` in AI05 shows the SDK side. To add it, the port needs document blocks, and the `Turn` needs citations, which is a real design change. Remember that citations don't combine with structured outputs.

Cost and limits

Decision	Effect
`TOP_K` of 4 here	Small on purpose in this lab. Tune it with evals (AI05 L6)
No model call for no results	Saves money, and gives a faster answer
Same message for "absent" and "not permitted"	Doesn't reveal that a document exists

Why the obvious fixes fall short

Tempting move	Why it fails	Better
Filter after search	Too few results, and restricted text got fetched	`WHERE` clause
Tell the model what the user may see	The model has read it already	Never retrieve it
Search with no permission when the directory's down	Leaks	Fail closed
Spring AI's `QuestionAnswerAdvisor` as is	Handy, with defaults to review (AI07 L3)	Check filtering and prompts
Testing RAG only with a live model	Slow, costly, and hides logic bugs	Ports and scripts, plus separate evals

How the field approaches it

Point	Source
Filtered approximate search and iterative scans	pgvector README (AI04 L3)
Hybrid search and rank fusion	AI04 L3, Anthropic's Contextual Retrieval
Documents first, question last	Claude prompting guidance (AI03 L2)
Citations give valid pointers	Citations docs (AI05 L4)

The production architecture

This is a recommended reference design, not a specific company's system. AI05 L7 covers the wider design, and ingestion.

Baseline: vector search, then a filter.

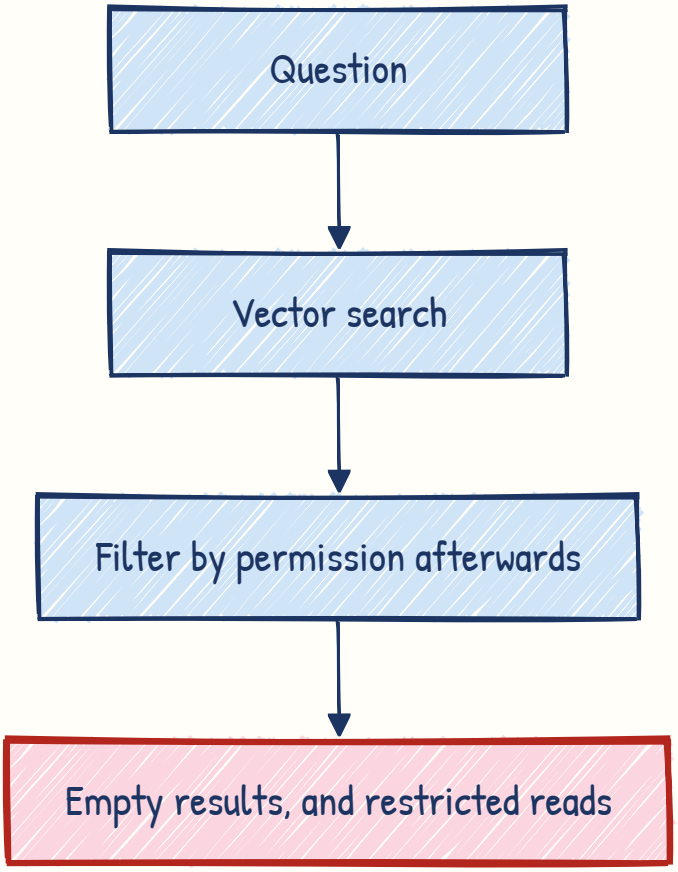


Figure 2. Baseline. Filtering after search can leave too little, and restricted text has already been read.

Improved: filter in SQL, with two rankings. New problem: directory calls add latency and load, and a stale cached group list can over-grant.

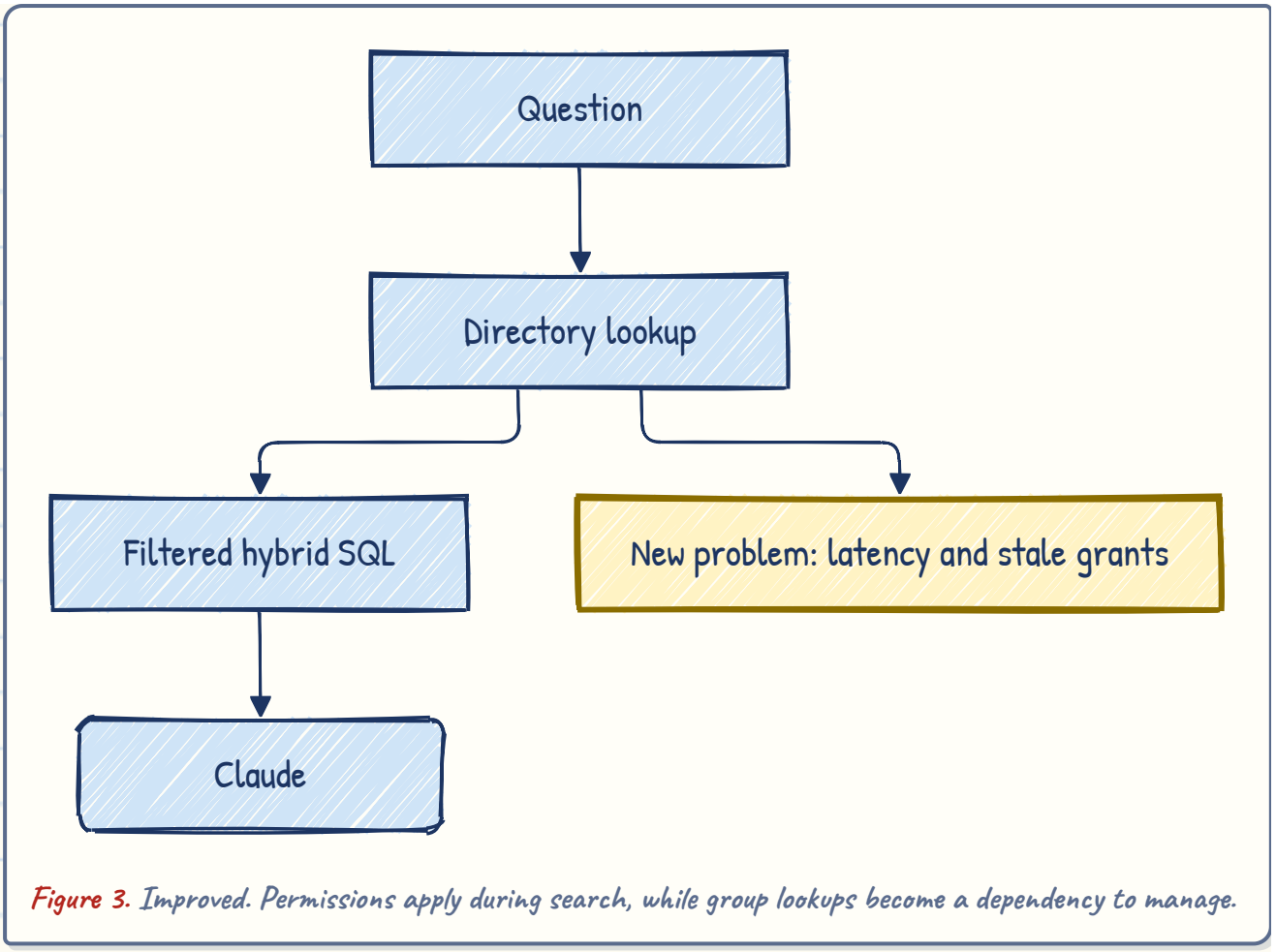


Figure 3. Improved. Permissions apply during search, while group lookups become a dependency to manage.

Production: short-lived group cache and read-time checks.

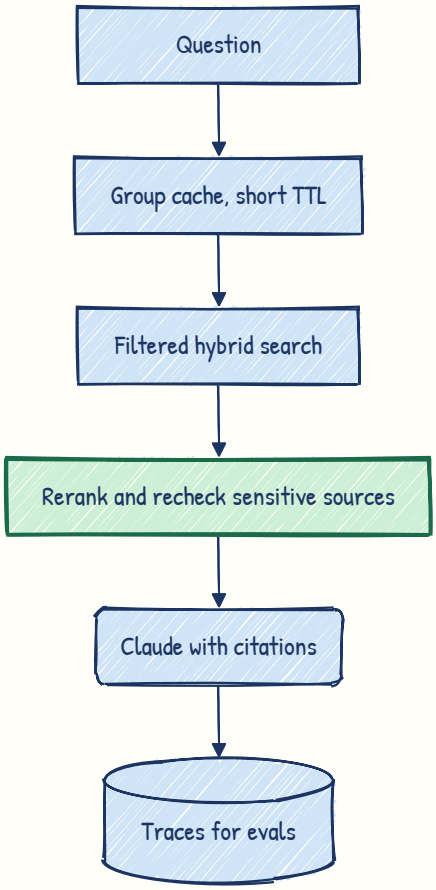


Figure 4. Production. Groups come from a short-lived cache, sensitive sources get rechecked at read time, and results are reranked before the model call.

Component	Why it exists	How it communicates
Group cache	Avoids a lookup per question	Short TTL, never serving when expired and the directory is down
Chunk table	Vectors, text and groups together	SQL. Read replicas for scale
Reranker	Precision at the top	HTTP API
Traces	Evaluation (AI05 L6)	Async

Failures

Failure	Containment
Directory down	Fail closed
Database down	`UNAVAILABLE`, never answer from memory
Revoked access still cached	Short TTL, and read-time checks on sensitive sources
Too few filtered results	Iterative scans (AI04 L3)
Embedding model changed	Version guard (AI04 L1)
Prompt injection in a document	Treat as data, and no write tools here (AI09 L2)

Trade-offs

Decision	Gain	Cost
Ports	Fast tests, replaceable store	Interfaces to maintain
pgvector	One database	Limits at large scale (AI04 L4)
Own RAG code	Control	Spring AI or LangChain4j would be less code
Failing closed	Safe	Outage when the directory's down

When simpler is fine: one team, one permission level. Skip the directory, and even consider a cached prompt (AI05 L1).

Common mistakes

1. **Filtering after search.** Filter in the query.
2. **Failing open when the directory's down.** Fail closed.
3. **Calling the model with no evidence.** It may answer from memory.
4. **Different messages for "absent" and "denied".** That reveals existence.
5. **Testing retrieval quality with a fake.** A fake tests logic only.
6. **Building SQL arrays by joining strings.** Use the driver's array support.
7. **Assuming unrun SQL works.** Test on your Postgres and pgvector versions.

What to say in an interview

"I'd put permissions first and fail closed. Groups are a required argument to search, so nobody can forget, and the filter is in the SQL. No evidence means no model call. Those decisions are all unit-testable with a script and an in-memory store. Retrieval quality isn't, so that gets a separate evaluation on real data, by segment."

Certification lens

How this lesson maps to CCAR-F

Not tested.

These are original practice questions for understanding, not real or leaked exam questions.

Q Q1 · Code analysis · Medium. `repo.search(query, 20).stream().filter(c -> user.canRead(c)).toList()`. Give two problems, and a fix.

A Restricted chunks are fetched before filtering, and a user with narrow access may see nothing, as all 20 might be filtered out. Filter in the query, by passing the user's groups to it. Also make groups a required parameter, so no caller can skip them.

Q Q2 · Testing · Medium. Which tests need a real Postgres, and which don't? (a) Fail closed on directory failure. (b) The SQL's filter returns only permitted rows. (c) Prompt has documents first. (d) Iterative scans return enough rows under a narrow filter.

A (a) and (c) don't: they're logic tests with fakes. (b) and (d) do, since they depend on real SQL and index behavior. Run them with a real container in CI, which this environment couldn't do.

Q Q3 · Best decision · Hard. To add citations, a teammate suggests asking the model to write "[source 1]" in its text. What are the options, and what would you pick?

A Prompt-based citation can invent sources. The Citations API returns valid pointers, though not proof of support, and it needs document blocks, so extend the port. Pick the API, verify support by sampling, and note it can't combine with structured outputs. If you stay with prompt-based, verify excerpts in code.

Revision - Lesson 5

→ Most RAG service work is decisions, not model calls.

→ Check permissions first, and fail closed.

→ Make groups a required search argument, and filter in SQL.

→ No evidence means no model call.

→ Same message for absent and denied.

→ Fuse two rankings with RRF.

→ Documents first, question last, with sources and dates.

→ PgVectorSearch here is compiled, and not run.

→ Fakes test logic. Retrieval quality needs real evals.

→ Citations need a port change, and can't combine with structured outputs.

Your notes